```
    return 0;
}

ssize_t write(struct file *filp, const char *buff,
size_t count, loff_t *offp)
{
    printk(KERN_INFO "Inside Write\n");
    return 0;
}

struct file_operations fops = {
    .read = read,
    .write = write,
    .open = open,
    .release = release
};

int schd_init (void)
{
    int ret;
    struct device *dev_ret;
    if ((ret = alloc_chrdev_region(&dev, FIRST_
MINOR, MINOR_CNT, "wqd")) < 0)
    {
    printk("Major Nr: %d\n", MAJOR(dev));

    cdev_init(&c_dev, &fops);

    if ((ret = cdev_add(&c_dev, dev, MINOR_CNT))
< 0)
    {
        unregister_chrdev_region(dev, MINOR_CNT);
        return ret;
    }
    if (IS_ERR(cl = class_create(THIS_MODULE,
"chardrv")))
    {
        cdev_del(&c_dev);
        unregister_chrdev_region(dev, MINOR_CNT);
        return PTR_ERR(cl);
    }
    if (IS_ERR(dev_ret = device_create(cl, NULL,
dev, NULL, "mychar%d", 0)))
    {
        class_destroy(cl);
        cdev_del(&c_dev);
        unregister_chrdev_region(dev, MINOR_CNT);
        return PTR_ERR(dev_ret);
    }
    return 0;
}

void schd_cleanup(void)
{
    printk(KERN_INFO " Inside cleanup_module\n");
    device_destroy(cl, dev);
    class_destroy(cl);
    cdev_del(&c_dev);
    unregister_chrdev_region(dev, MINOR_CNT);
}

module_init(schd_init);
module_exit(schd_cleanup);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Pradeep Tewani");
MODULE_DESCRIPTION("Waiting Process Demo");
```

The example shown above is a simple character driver demonstrating the usage of *schedule()* API. *schd_init()* and *schd_cleanup()* are the initialisation and exit functions for the driver, respectively. Apart from these, there are open, close, read and write functions, which get invoked when the process performs the respective operations on the device file. One point worth noting is that in the *read* function, we invoke the *schedule* function. This in turn means that whenever the process performs the *read* operation on the device file, it would be voluntarily giving up the process. Let's compile this program as a module and load it using the *insmod* command. Shown below is the sample run:

```
$ insmod wait.ko
Major Nr: 250
$ cat /dev/mychar0
Inside open
Inside read
Scheduling out
Woken up
```

Note: The above message won't directly come on the shell. We will have to execute the *dmesg* command to view these messages.

Upon loading the driver, we will get the device file with a name *mychar0* in */dev* directory. Then, we use the *cat* command to perform the *read* operation. This, in turn, will invoke the *schedule* function. One thing that we notice from the sample run is that invoking the *schedule()* doesn't really block the process, instead it immediately comes out of the wait. Why is it so? Is there something wrong with the schedule function? Hold on… if you recall the functionality of *schedule*, it gives up the processor, but doesn't move the process out of the run queue. And as long as the process is in the run queue, it is scheduled to run during the next scheduling cycle. This means that in order to block the process, it needs to be moved out of the run queue. So, how do we move the process out of the run queue? This requires the